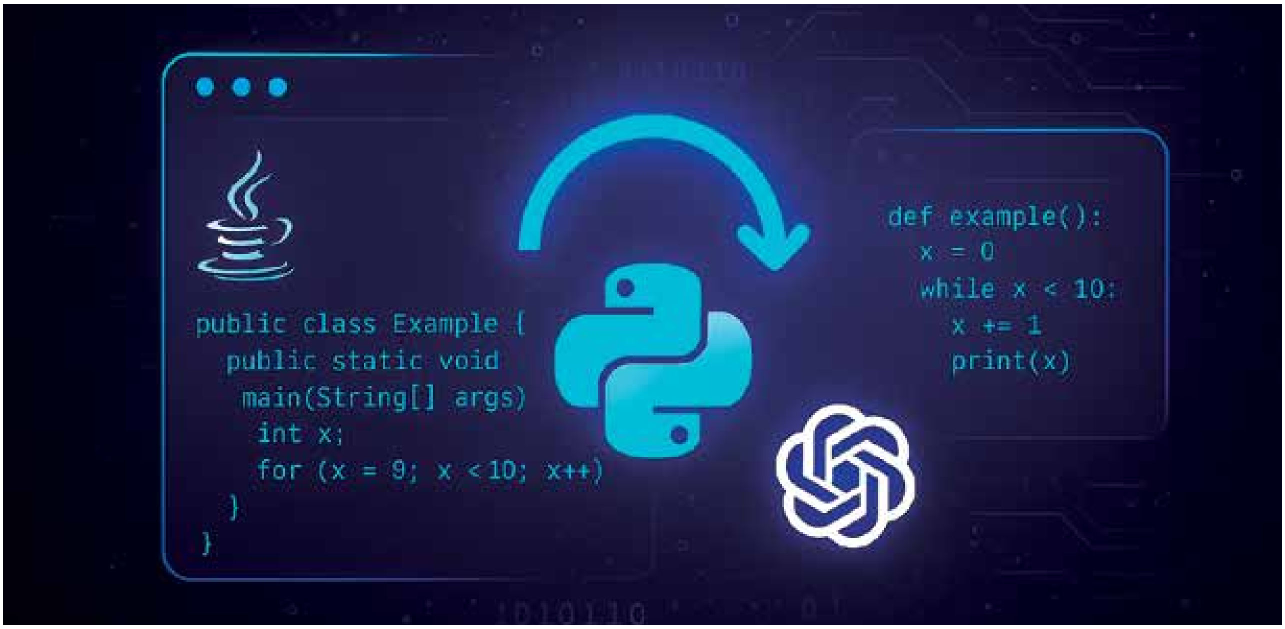
How to Automate Java Code Modernisation

This short guide illustrates that automating Java code modernisation with Python and OpenAI API is not just possible—it's remarkably effective.



Modernising legacy Java code is a crucial but often labour-intensive task. Whether you're updating code to the latest Java standards, refactoring for performance, or migrating projects to frameworks such as Spring Boot, software teams face a formidable challenge. Fortunately, advances in generative AI—especially models like OpenAI's GPT—offer a new way to automate this process. With Python's ease of scripting and integration, it becomes possible to build a tool that prompts the OpenAI API with Java code snippets and receives modernised versions in response.

Let's now walk through building a Python script that automates Java code modernisation using OpenAI API calls. We'll explore the architecture, explain each component of the script, and provide practical examples, including prompts for upgrading Java code or migrating to Spring Boot. Code snippets and sample outputs will help you clearly understand how to build and use this automation.

Why automate Java code modernisation?

Before we dive into the technical implementation, let's briefly consider why automation is valuable.

- ***Consistency:*** Automated modernisation ensures standardised upgrades across large codebases.
- ***Efficiency:*** Automation saves developer time, especially for repetitive tasks.
- ***Scalability:*** Large-scale migration projects can be handled with minimal manual intervention.
- ***Quality:*** AI-based suggestions can reveal best practices you may overlook.

Designing the Python script

To follow this guide, you need:

- Python 3.7 or higher
- OpenAI API access and key
- The OpenAI Python package (pip install openai)

The core idea is to write a Python script that:

- Accepts legacy Java code (from a file or string).
- Constructs a prompt instructing OpenAI to modernise the code.
- Sends the prompt to OpenAI's API using Python.
- Receives and displays the modernised Java code

Let's break down the script and understand each part.